

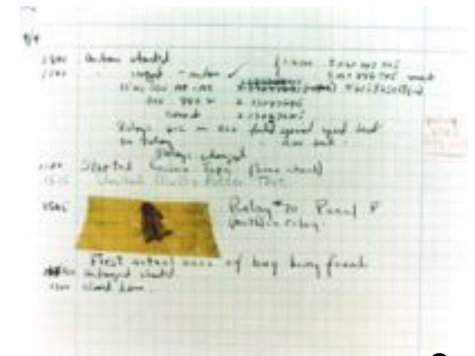
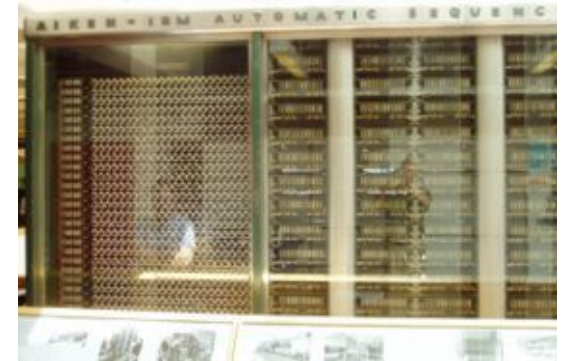
Introduction to Software Testing

Chris Meudec

2018-2038

What is a computer bug?

- In **1947** Harvard University was operating a room-sized computer called the Mark II.
 - mechanical relays
 - glowing vacuum tubes
 - technicians program the computer by reconfiguring it
 - technicians had to change the occasional vacuum tube.
- A moth flew into the computer and was zapped by the high voltage when it landed on a relay.
- Hence, the first computer bug!



Detour...

- Software as a maze

[see my short research presentation](#)

Bugs a.k.a. ...

- Defect
- Fault
- Problem
- Error
- Incident
- Anomaly
- Issue
- Variance
- Failure
- Inconsistency
- Product Anomaly
- Product Incidence
- Feature :-)

and the best...

- The Irish Aviation Authority invented a new description in October 2018:

"irregular software occurrence"

from an organisation in charge of a safety critical software operation it does not inspire confidence... [[link](#)]

Defective Software

- We develop programs that contain defects
 - How many? What kind?
- Hard to predict the future, however...
it is highly likely, that software we will develop in the future will not be significantly better.

Sources of Problems

- **Requirements Definition:** Erroneous, incomplete, inconsistent requirements.
- **Design:** Fundamental design flaws in the software.
- **Implementation:** Mistakes in chip fabrication, wiring, programming faults, malicious code.
- **Support Systems:** Poor programming languages, faulty compilers and debuggers, misleading development tools.

Sources of Problems (Cont'd)

- **Inadequate Testing of Software:**
Incomplete testing, poor verification, mistakes in debugging.
- **Evolution:** Sloppy redevelopment or maintenance, introduction of new flaws in attempts to fix old flaws, incremental escalation to inordinate complexity.

Adverse Effects of Faulty Software

- **Communications:** Loss or corruption of communication media, non delivery of data.
- **Space Applications:** Lost lives, launch delays.
- **Defense and Warfare:** Misidentification of friend or foe.

Adverse Effects of Faulty Software (Cont'd)

- **Transportation:** Deaths, delays, sudden acceleration, inability to brake.
- **Safety-critical Applications:** Death, injuries.
- **Electric Power:** Death, injuries, power outages, long-term health hazards (radiation).

Adverse Effects of Faulty Software (Cont'd)

- **Money Management:** Fraud, violation of privacy, shutdown of stock exchanges and banks, negative interest rates.
- **Control of Elections:** Wrong results (intentional or unintentional).
- **Control of Jails:** Technology-aided escape attempts and successes, accidental release of inmates, failures in software controlled locks.
- **Law Enforcement:** False arrests and imprisonments.

What about AI?

Bug in Space Code

- Project Mercury's FORTRAN code had the following fault:

`DO I=1.10`

instead of `DO I=1,10`

- The fault was discovered in an analysis of why the software did not seem to generate results that were sufficiently accurate.
- The erroneous `1.10` would cause the loop to be executed exactly once!

Military Aviation Problems

- An F-18 crashed because of a missing exception condition:
`if... then...` without the `else` clause that was thought could not possibly arise.
- In simulation, an F-16 program bug caused the virtual plane to flip over whenever it crossed the equator, as a result of a missing minus sign to indicate south latitude.

Therac-25 Radiation “Therapy”

- In Texas, 1986, a man received between 16,500-25,000 rads in less than 1 sec, over an area of about 1 cm.
- He lost his left arm, and died of complications 5 months later.
- In Texas, 1986, a man received at least 4,000 rads in the right temporal lobe of his brain.
- The patient eventually died as a result of the overdose.

Therac-25 Radiation “Therapy” (Cont’d)

- In Washington, 1987, a patient received 8,000-10,000 rads instead of the prescribed 86 rads.
- The patient died of complications of the radiation overdose.

Task 10 mins

- Find one different famous bug
- In a Google Doc:
 - put URL(s) to that bug
 - Describe how you think it could have been prevented in 5 lines.
 - put your Google Docs in this Shared Folder

Specification

- You have to know precisely how your software should behave before you can say if it has a bug.
- Problem: in Agile processes, specifications are usually informal (unlike when using formal specifications)

Specification

- A *specification* defines the product being created and includes:
 - Functional requirements that describes the features the product will support. E.g., on a word processor
 - Save, print, check spelling, change font, ...
 - Non-functional requirements are constraints on the product. E.g,
 - Security, reliability, user friendliness, platform, resources usage (time, memory, bandwidth...)
 - Often harder to test

A software bug occurs when at least one of these rules is true

- The software does not do something that the specification says it should do.
- The software does something that the specification says it should not do.
- The software does something that the specification does not mention.
- The software does not do something that the product specification does not mention but should.
- The software is difficult to understand, hard to use, slow... (non-respect of non-functional requirements)

In Waterfall, most bugs are not because of mistakes in the code ...

- Specification ($\sim$ 55%)
 - Time to think: how are these 'bugs' detected?
- Design ($\sim$ 25%)
- Code ($\sim$ 15%)
- Other ($\sim$ 5%)

Relative cost of bugs

“bugs found later cost more to fix”

- Cost to fix a bug increases exponentially (10^x)
 - i.e., it increases tenfold as time increases
- E.g., a bug found during specification costs \$1 to fix.
- ... if found in design cost is \$10
- ... if found in code cost is \$100
- ... if found in released software cost is \$1000

Bug Free Software

- Software is often in the news for the wrong reason
- Human beings make mistakes, especially when asked to create complex artifacts such as software systems.
- Studies show that even good programs have 1-3 bugs per 100 lines of code.
- People who claim that they write bug-free software probably haven't programmed many deployed applications.

Misunderstandings

- Software testing is the least well practiced activity in software development:
 - a lot of wasted efforts
 - a lot of worthless testing...
- Non software developers' (managers, decision-makers etc.) understanding of software testing is shockingly poor:
 - *"It was running the data in parallel with the contractor, the minister said"* RTE News [Independent review of calculated grades could be completed today](#)
 - statistical model checking is not defect testing: it is validation... [Hence [Ireland needs a CSO](#)]

Quality

The amount of care that must go into releasing a commercial software is huge:

- The “*running the code a few times to see it works*” approach is not suitable in the real world.
- Users and buyers of software products expect your software to work for them right away (their business increasingly depends on software) in their own environment: they rightly have no patience for installation problems, bugs, poor interfaces, unreliability, security holes etc.

Quality

The amount of effort necessary to transform a basic software that has worked a few times into a commercial product is at a minimum 100% the initial effort:

- in other words at least 50% of the development effort of a commercial software is usually spent ensuring that the software will work as expected for the intended users (for student projects, research projects etc. the necessary effort would be much higher)

Defective Software

Why is software the last product that we sell 'as-is', in other words, without a guarantee?

- How long will it be before software companies are sued for loss of life (safety breach)? or loss of business information (security breach)? or plain bugs (unreliability)?

Goal of a software tester

- ... to *find* bugs
- ... as *early* in the software development processes as possible
- ... and make sure they get *fixed*.

Dev Community Response

- Better processes to ensure that *we are building the right thing*.
- Technology to help us ensure that we *built the thing right*.
- Verification is an area where significant software process improvements can be made.
- Trend is towards using mixed Verification and Validation methods **and** trying to prevent bugs in the first place

Software Testing

- Software testing is a critical element of software quality assurance and represents the ultimate review of:
 - specification
 - design
 - coding
- Traditional Software life-cycle models (*e.g.*, waterfall) frequently include software testing as a separate phase that follows implementation!

Software Testing (Cont'd)

- Contrary to traditional life-cycle models, testing is an activity that must be carried out throughout the life-cycle: part of everyday development
- **Agile Motto:** test early, test often (test realistically)
- DevOps influences

Terminology

- **Test case**: Inputs to test the program and the predicted outcomes (according to the '*specification*'). Test cases are formal procedures:
 - inputs are prepared
 - outcomes are predicted
 - tests are documented
 - commands are executed
 - results are observed and evaluated

Note: all of these steps are subject to mistakes.

- When does a test “succeed”? “fail”?

Terminology

- **Test suite**: A collection of test cases
- **Testing oracle**: A mechanism (a program, process, a person, or body of data) which helps us determine the expected outcome.
 - Oracles are often defined as a set of input/expected outcome pairs.
 - An oracle can be a person (e.g. an aircraft pilot in the case of an automated flying system, or a mathematical formula in the case of loan system)

Terminology

- **Regression Test Suites:** Test software using the test suites developed for previous versions of the same software.
- Regression testing is the activity of re-testing modified software.
- Necessary because it is common to introduce new bugs when modifying existing code to either correct an existing problem or otherwise enhance the program.
- For some systems, regression testing can be fairly expensive.
 - only manageable if test cases are available and they can be executed automatically. (e.g. as part of a DevOps pipeline)

Terminology

- **Coincidental correctness**: A program is said to be coincidentally correct if the test results in the expected outcome, even though the program performs the incorrect computation.

Example: A program is to calculate $y = x^2$

It is incorrectly programmed as $y = 2 * x$, and it is tested with the input value $x = 2$

Expected Outcome

- Sometimes, specifying the expected outcome for a given test case is very difficult
 - For some applications we might not know what the outcome should be.
 - For other applications the developer might have a misconception
 - Finally, the program may produce too much output to be able to analyze it in a reasonable amount of time.
 - In general, this is a fragile part of the testing activity, and can be very time consuming.

Expected Outcome

- In practice, this is an area with a lot of hand-waving.
- When possible, automation should be considered as a way of specifying the expected outcome, and comparing it to the actual outcome.

Goals of Testing

- Discover and prevent bugs.
- The act of designing tests is one of the best bug preventers known.
 - Test, then code philosophy
- The thinking that must be done to create a useful test can discover and eliminate bugs in all stages of software development.
- However, bugs will always slip by, as even our test designs will sometimes be buggy.

The Significance of Testing

- Most widely-used activity for ensuring that software systems satisfy the specified requirements.
- Consumes substantial project resources.
Some estimates: ~50% of development costs

Testing Phases

- For Verification :
 - **Unit testing** : testing of individual code methods.
 - **System testing** : testing the complete system from the GUI or API.
- For Validation :
 - **Acceptance testing** : testing using tests provided by the client to check that the system satisfies requirements. Sometimes called alpha testing.
 - **Beta testing** : usually large scale testing by users in pre-release configuration.

Limitations of Testing

- *Exhaustive testing* is not practical even for the simplest programs. WHY?
- Even if we “exhaustively” test all execution paths of a program, we cannot guarantee its correctness.
 - The best we can do is *increase our confidence!*
- “*Testing can show the presence of bug, not their absence.*” Edsger W. Dijkstra

Limitations of Testing

- Which testing method is the best at finding errors? The debate is still often focused on black-box vs. white-box testing.
- For white box testing:
 - Defects may manifest themselves when particular path combinations arise (even if all program statements have been executed at least once)
 - Thus, **there is no simple relationship between structural testing coverage measurements and the number of remaining bugs in a program.** (even if 100% path coverage is achieved). In an experiment, black-box testing was found to be more effective than white-box testing in discovering defects
- In another experiment random testing was found to be the most effective!!
 - The debate has moved on a bit: recognition that white and black box are **complementary**
 - Also random testing is more valuable for validation than verification (i.e. to increase confidence)
- To further blur the situation, static code reviewing is often less expensive and more effective in discovering program faults (at least at the class level)

Limitations of Testing

- A successful test is a test which discovers one or more errors :
 - psychological mind set problem: what is a successful test run?
 - idea of independent testing (not performed by the developers).
- Unit Testing is usually the responsibility of the programmers (because to ensure code coverage requires deep understanding of the code). But can be very dangerous:
 - **psychologically programmers may not be inclined to construct tests that will show defects in their own work;**
 - **programmers may have misunderstood the specification: may fail to recognise an incorrect output.**
- Independent testing should increase the bug detection rate but is much more expensive

Limitations of Testing

- Very difficult to test non-functional requirements
 - e.g. run-time (Worst Case Execution Time estimates are important for real-time systems)
FYI [see Eileen Dillon's brilliant MSc thesis](#)
 - e.g. memory usage (important for embedded systems)
 - e.g. usability (important for human interactions)

Limitations of Testing

- Tests designed to discover system defects against the specification (suited for verification)
- A successful defect test is one which reveals the presence of defects in a system.
- Should be used in conjunction with static verification.

Limitations of Testing

- When to stop testing?
- We have no way of knowing how many errors are left in the program under test after **any** form of testing...
- Some common testing stopping criteria:
 - when the management says so;
 - when the human tester decides so;
 - when the money runs out;
 - when the time runs out.
- **A better stopping criterion:**
 - when the likely resources (time, money, effort) involved in finding another error (weighted by seriousness) is too great (based on statistics).
 - See diagram...

